



Department of Computing Sciences	
Academic Year	2026-2027
Semester	☒ Fall ☐ Spring ☐ Summer
Course Code - Name	GCIS-123 – Software Development and Problem Solving II
Instructor	
Exam	☒ Assignment ☐ Quiz/Test ☐ Mid ☐ Final ☐ Other Assignment 1
Duration	~1 week
Date	Sep 23,2026 – October 1, 2026

- No AI tools are allowed to assist in generating any part of the solution of Assignment 1.
- The assessment is a group activity and is published as a homework, which has to be submitted on MyCourses. Every group will be assessed by viva and class presentation.
- Any code used from unapproved sources, and any code concept that we did not cover in class before the announcement date, will be marked as 0.
- Any member or group unable to present on the designated date(s) will be marked 0. No extended date will be assigned to the group or member for viva and/or presentation.
- All students must be able to answer questions on every part of the submission, regardless of who developed which part.

Group No.: Group 3

Student Names: Ananya Yellamilli, Hafsa Sultana, Ilham Asif

CLOs:

1. Utilize the basic constructs of a selected contemporary high-level programming language.
2. **Apply problem-solving skills to develop solutions for computing problems of varying complexity.**
3. Implement algorithms and data structures to solve computing problems of different difficulty levels.
4. Demonstrate fundamental software engineering concepts, including requirements analysis, software design, testing, and incremental development.

☒ Check this box if this assessment maps to ONE and only ONE CLO

CLO	Max Grade	Achieved Grade (Enter name and ID for each student in a group only when students receive different grade)			
CLO 2	100	Student 1 ID1	Student 2 ID2	Student 3 ID3	Student 4 ID4
		Ananya Yellamilli 730001402	Hafsa Sultana 730003960	Ilham Asif 445009308	

Assignment 1: HomeSense – Smart Home Energy Monitor

Smart homes contain connected devices that continuously consume electricity. A smart home system can monitor this consumption and identify devices that may require attention.

In this assignment, you will develop HomeSense, a Python-based smart home energy monitoring program. HomeSense will receive energy readings from different household devices and analyze them to determine whether their consumption is within an expected range.

The assignment begins with a relatively simple analysis of one device reading and progressively develops into the analysis of multiple readings and the generation of a smart-home energy report.

The emphasis of this assignment is problem solving. You are given the problem, requirements, and reference data. You must decide how to decompose the problem, design your functions, construct your conditions, process repeated readings, and test your solution.

No physical IoT hardware, external API, or AI service is required.

Objectives

Upon successful completion of this assignment, students should be able to:

- Break a real-world computing problem into smaller programming problems.
- Design functions with appropriate parameters and return values.
- Apply conditional statements to solve problems involving multiple possible outcomes.
- Use loops to process repeated data.
- Combine multiple functions into a larger solution.
- Develop and justify rules for classifying sensor readings.
- Identify boundary and unusual cases.
- Write basic pytest tests using assertions.
- Explain the problem-solving decisions made during implementation.

Technology Requirements

- **Programming language:** Python
- **Testing framework:** pytest
- **External libraries:** Not permitted.
- **IoT hardware:** Not required.
- **External APIs:** Not permitted.
- **AI tools/services:** Not permitted.
- *The solution must use only programming concepts covered in class before the assignment announcement.*

Scenario: HomeSense

Imagine that a smart apartment contains several connected electrical devices.

The HomeSense system receives an energy reading for a device. The reading represents the amount of energy consumed during a defined monitoring period and is measured in **kWh**.

The system should be able to determine whether a reading is:

- **Normal**
- **High**
- **Critical**

and provide appropriate feedback to the homeowner.

However, different devices naturally consume different amounts of energy. For example, it would not be reasonable to apply the same energy threshold to an LED light and an air conditioner.

HomeSense therefore uses the following reference ranges.

Reference Data: Normal Operating Ranges

The following table defines the expected **normal operating range for one monitoring reading**.

Device	Normal Energy Consumption
LED Light	0.01 – 0.10 kWh
Television	0.05 – 0.50 kWh
Refrigerator	0.10 – 1.50 kWh
Washing Machine	0.30 – 2.50 kWh
Air Conditioner	0.50 – 5.00 kWh

These ranges are provided as the common baseline for all groups.

Important

The table tells you what is considered **normal**. It does **not** give you the complete algorithm for deciding whether an abnormal reading should be classified as **High** or **Critical**.

Your group must design this logic.

For example, you must decide how your program distinguishes between:

- Normal
- High
- Critical

when a reading is above the normal range.

Your decision must be:

- logically consistent,
- implemented through Python conditions,
- applicable to different devices,
- documented in your submission,
- and explainable during the viva.

Different groups may therefore implement different reasonable approaches.

Task 1: Analyze a Single Device Reading

Develop functions that analyze one energy reading.

1.1 Device Range Function

Create a function that receives a device name and returns its normal operating range.

For example, conceptually:

device → minimum normal value, maximum normal value

The function should allow the rest of the program to determine the appropriate range for the selected device.

You must decide how your function handles an unknown device.

1.2 Energy Status Function

Create a function that receives:

- device
- energy consumption

and returns one of:

- Normal
- High
- Critical

The function must use the reference ranges provided in the assignment.

Problem-solving requirement

The exact rule separating **High** from **Critical** is intentionally not provided.

Your group must:

1. Decide on a rule.
2. Implement the rule.
3. Explain why the rule is reasonable.

1.3 Attention Function

Create a function that determines whether a device requires attention.

The function should return an appropriate Boolean result:

True

False

The function should be based on the status produced by your analysis.

Task 2: Estimate Energy Cost

HomeSense should also provide an estimated electricity cost.

Create a function that receives:

- energy consumption in kWh
- electricity rate in AED/kWh

and returns the estimated cost.

For example:

Energy consumption = 4.0 kWh

Electricity rate = 0.30 AED/kWh

Estimated cost = 1.20 AED

The function must return the calculated result rather than simply printing it.

You must also consider appropriate handling of invalid values, such as negative energy consumption.

Task 3: Generate Smart Feedback

Create a function that receives information about a device and its energy reading and returns an appropriate recommendation.

For example:

Device: Air Conditioner

Energy: 7.5 kWh

Status: High

Recommendation:

Check the operating duration and temperature settings.

Your program must provide **different feedback for different situations**.

At least three different recommendation scenarios must be supported.

The exact wording and decision rules are part of your group's design.

Task 4: Analyze Multiple Smart-Home Readings

A real smart-home system does not receive only one reading.

The following sample readings are provided for testing your program:

#	Device	Energy (kWh)
1	LED Light	0.06
2	LED Light	0.18
3	Television	0.32
4	Television	1.20
5	Refrigerator	0.80
6	Refrigerator	2.20
7	Washing Machine	1.40
8	Washing Machine	4.50
9	Air Conditioner	2.80
10	Air Conditioner	7.50

Your program must use a **loop** to process the readings.

You must not write separate processing code for every reading.

For every reading, determine:

- Device
- Energy consumption
- Status
- Whether attention is required
- Estimated cost

Task 5: HomeSense Summary

After processing all readings, the program must calculate and display at least:

- Total number of readings
- Total energy consumption
- Total estimated cost
- Number of Normal readings
- Number of High readings
- Number of Critical readings
- Number of readings requiring attention
- Device with the highest individual consumption

The output should be based on the actual readings processed by the program.

Task 6: HomeSense Report

Create a readable report similar to the following. The exact formatting is up to your group.

===== HomeSense Report =====

Readings analyzed: 10

Normal: 6

High: 3

Critical: 1

Readings requiring attention: 4

Total energy: 21.50 kWh

Estimated cost: AED 6.45

Highest consumption:

Air Conditioner - 7.50 kWh

- The numbers shown on the left (in blue) are illustrative.
- Your program must calculate its own results

Task 7: Boundary and Unusual Cases

A good solution should not only work for ordinary readings.

Your program must consider at least **four boundary or unusual cases**.

Examples include:

- Energy exactly at the lower normal limit.
- Energy exactly at the upper normal limit.
- Energy slightly above the normal range.
- Energy significantly above the normal range.
- Zero energy consumption.
- Negative energy consumption.
- Unknown device.

Your group must identify the cases you tested and explain the expected behaviour.

Task 8: pytest Testing

Create pytest tests for your functions.

At least **8 meaningful test cases** are required.

Your tests must include:

- A normal reading.
- A reading at a boundary.
- A high reading.
- A critical reading.
- An energy-cost calculation.
- An attention decision.
- An unusual/invalid input.
- Another case selected by your group.

Use assertions to verify expected results.

For example:

```
def test_energy_cost():
    assert calculate_cost(4.0, 0.30) == 1.20
```

Your tests must test the **logic of your functions**, rather than merely checking whether the program executes without errors.

Task 9: Problem-Solving & Design Explanation

Your group must provide a short design explanation addressing the following.

A. Problem decomposition

How did you break the HomeSense problem into smaller functions?

B. Decision making

How did you decide the difference between: (?)

- Normal
- High
- Critical

C. Boundary cases

What happens when a reading is exactly equal to the upper normal limit?

D. Different devices

Why should an air conditioner and an LED light not use the same energy threshold?

E. Multiple readings

How did your solution change when moving from analyzing one reading to analyzing many readings?

F. Testing

Which test cases were most important for verifying your solution, and why?

Required Deliverables

Each group must submit:

1. **Python source code**
2. **pytest test file(s)**
3. Sample input/readings used by the program
4. Screenshot of the HomeSense program output
5. Screenshot showing successful pytest execution
6. A short README explaining how to run the program
7. A short **Problem-Solving & Design Explanation**
8. A short statement documenting the group's **High/Critical classification rule**

Grading Rubric – 100%

Evaluation Area	Weight
Problem decomposition and function design – appropriate functions, parameters and return values	20%
Conditional problem-solving – logical classification and handling of different cases	20%
Loop-based processing – correct processing of multiple readings	15%
Solution integration – functions work together to produce the complete HomeSense solution	15%
pytest testing – meaningful assertions, boundary and unusual cases	15%
Problem-solving explanation – justification of decisions and approach	10%
Code readability and presentation including doc strings	5%
Total	100%

Academic Integrity

- All submissions must comply with the RIT Academic Integrity Policy.
 - All code must be the original work of the group.
 - **No AI tool may be used to generate, complete, modify, debug, explain, or refactor any part of the submitted solution.**
 - Code copied or adapted from unapproved sources is not permitted.
 - During the viva, students must be able to explain the submitted solution and may be asked to modify or extend part of their own code.
 - If a student cannot explain a submitted solution or demonstrates that the work was not independently developed by the group, marks may be deducted from the assessment.
- The purpose of the viva is to verify working/understanding of the submitted work, not simply the ability to produce a working program.**